

16/16, clean. Decoder, encoder, and demux are all done and verified correctly. Day 5 Sessions 1 and 2 are closed out.

DAY 5 — SESSION 3: COMPARATORS & ARITHMETIC CIRCUITS

BLOCK 1: CONCEPT (10–12 min)

Why This Session Matters Most

Everything before this — MUX, decoder, encoder, DEMUX — was about **routing and selecting** data. Nothing computed anything.

This session is different. Comparators and adders **compute**. This is the direct precursor to your ALU's arithmetic unit, which you build tomorrow.

Part A — Comparators

A comparator takes two numbers and tells you their relationship: equal, greater, or less.

The simplest comparator — equality check

`out = (a == b);`

That's it. One line. `==` already does this. No custom logic needed for simple equality.

Magnitude comparator — full version

Outputs needed:

`eq : a == b`

`gt : a > b`

`lt : a < b`

This matters for your ALU's **CMP operation**, and for the **Zero (Z)** and **Negative (N)** flags. When you do SUB in the ALU, you don't just get a result — you check if that result is zero, negative, etc. That check is a comparator, hidden inside the flag logic.

Signed vs Unsigned Comparison — Critical Concept

This is where people get hurt in real ALU design.

4-bit unsigned: `1111 = 15`

4-bit signed: 1111 = -1 (two's complement)

Same bit pattern. Different meaning depending on interpretation.

If you compare `4'b1111 > 4'b0001` as **unsigned**: `15 > 1 → true`. If you compare the same bits as **signed**: `-1 > 1 → false`.

Verilog's `>` operator on reg/wire (unsigned by default) does **unsigned comparison** unless you explicitly declare signed types or cast with `$signed()`.

```
wire signed [3:0] a = 4'b1111; // -1
```

```
wire signed [3:0] b = 4'b0001; // 1
```

```
wire result = (a > b); // signed comparison: false (-1 is not > 1)
```

This is exactly why your ALU needs a dedicated **Negative flag (N)** — checking the MSB of the result tells you the sign, and your CMP/SUB operations must be aware of signed interpretation for correct flag behavior.

Part B — Adders

Half Adder

Adds two single bits. No carry-in.

a b sum carry

0 0 0 0

0 1 1 0

1 0 1 0

1 1 0 1

`sum = a XOR b`

`carry = a AND b`

Full Adder

Adds two bits **plus a carry-in**. This is the real building block — it's what gets chained to build multi-bit adders.

a b cin sum cout

0 0 0 0 0

a b cin sum cout

0 0 1 1 0

0 1 0 1 0

0 1 1 0 1

1 0 0 1 0

1 0 1 0 1

1 1 0 0 1

1 1 1 1 1

sum = a XOR b XOR cin

cout = (a AND b) OR (cin AND (a XOR b))

Ripple Carry Adder — N-bit Addition

Chain N full adders together. Each stage's cout feeds the next stage's cin.

a[3]a[2]a[1]a[0]

+ b[3]b[2]b[1]b[0]

FA3 ← FA2 ← FA1 ← FA0 ← cin=0

| | | |

sum3 sum2 sum1 sum0

In real RTL, you never manually chain full adders for a generic adder — you just write `sum = a + b;` and let the synthesis tool pick the best adder architecture (ripple carry, carry-lookahead, etc. depending on target). But you must **understand** what's underneath, because this is exactly how your ALU computes Carry (C) and Overflow (V) flags.

Carry vs Overflow — The Flag You Will Get Wrong If You Don't Understand This Now

This is the single most misunderstood part of ALU design. Pay close attention.

Carry (C): relevant for **unsigned** arithmetic. Did the result overflow the bit width?

4-bit unsigned: 1111 + 0001 = 10000

↑

carry out = 1

Result stored: 0000 (wrapped around)

Carry flag: 1 (tells you a wraparound happened)

Overflow (V): relevant for **signed** arithmetic. Did the sign of the result become wrong?

4-bit signed: 0111 (+7) + 0001 (+1) = 1000 (-8)

positive + positive = negative ← WRONG sign!

Overflow flag: 1 (result sign is impossible given the inputs)

The actual hardware rule for overflow on addition:

```
overflow = (a[MSB] == b[MSB]) && (result[MSB] != a[MSB]);
```

In words: if both operands have the **same sign**, but the result has a **different sign**, that's an overflow. Two positives can't make a negative. Two negatives can't make a positive. If they do — overflow.

This exact line of logic goes directly into your ALU on Day 6. I am not asking you to memorize it now — I am making sure you've seen it once, clearly, before you have to write it yourself.

Common Mistakes

Mistake 1: Using unsigned comparison when you meant signed

if (a > b) // unsigned by default — wrong if a, b represent negative numbers

Mistake 2: Confusing carry and overflow They are NOT the same flag, NOT the same condition, and NOT interchangeable. Carry is for unsigned overflow detection. Overflow is for signed sign-correctness detection. Your ALU needs both, separately.

Mistake 3: Computing carry by checking if sum < a

// WRONG and incomplete

```
carry = (sum < a);
```

This sort of works for addition by accident in simulation, but it is not how real hardware computes carry, and it breaks down once you also need subtraction's borrow logic. Always extend the bit width by 1 and read the extra bit — shown below.

BLOCK 2: CODE & PRACTICAL (10–12 min)

Design 1: Half Adder

```
// FILE: half_adder.v
module half_adder (
    input  wire a, b,
    output wire sum, carry
);
    assign sum  = a ^ b;
    assign carry = a & b;

endmodule
```

Design 2: Full Adder

```
// FILE: full_adder.v
module full_adder (
    input  wire a, b, cin,
    output wire sum, cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));

endmodule
```

Design 3: N-bit Adder with Carry-Out (THE pattern your ALU will use)

```
// FILE: adder_n.v
// Parameterized adder with explicit carry-out
// This is the actual pattern used inside the ALU arithmetic unit

module adder_n #(
    parameter WIDTH = 8
)(
    input  wire [WIDTH-1:0] a, b,
```

```

input wire      cin,
output wire [WIDTH-1:0] sum,
output wire      cout
);

// KEY TECHNIQUE: extend operands by 1 bit, add with extra width,
// then the top bit of the wide result IS the carry-out.
// This is the correct, synthesizable way to get carry — not sum<a tricks.
wire [WIDTH:0] result_wide;

assign result_wide = {1'b0, a} + {1'b0, b} + cin;

assign sum = result_wide[WIDTH-1:0]; // lower WIDTH bits = sum
assign cout = result_wide[WIDTH];    // extra bit = carry-out

endmodule

```

Why {1'b0, a} instead of just a?

If a and b are both WIDTH bits, a + b in Verilog still computes at WIDTH bits by default in some contexts and you can silently lose the carry. By explicitly extending to WIDTH+1 bits before adding, you **guarantee** the carry is captured in bit [WIDTH], with no ambiguity. This is the standard, safe RTL idiom — memorize this pattern, you will use it directly in the ALU.

Design 4: Subtractor with Borrow (using two's complement)

```

// FILE: subtractor_n.v

// Subtraction implemented as addition of two's complement
// This is EXACTLY how real ALUs implement SUB — same adder hardware, reused

module subtractor_n #(
    parameter WIDTH = 8
)(

```

```

input wire [WIDTH-1:0] a, b,
output wire [WIDTH-1:0] diff,
output wire      borrow // 1 = borrow occurred (a < b, unsigned)
);

wire [WIDTH:0] result_wide;

// a - b == a + (~b) + 1 (two's complement subtraction)
assign result_wide = {1'b0, a} + {1'b0, ~b} + 1'b1;

assign diff = result_wide[WIDTH-1:0];

// For subtraction, borrow is the INVERSE of carry-out
// If a >= b (unsigned), there's no borrow, and the adder produces a carry
assign borrow = ~result_wide[WIDTH];

endmodule

```

Why does subtraction reuse the adder?

This is one of the most important hardware facts in ALU design: **a real ALU does not have separate adder and subtractor circuits.** It has ONE adder. For subtraction, it inverts b and feeds in cin = 1. That's it — same hardware, different control signal. You will implement this exact trick in your Day 6 arithmetic unit.

Design 5: Magnitude Comparator (Unsigned)

```

// FILE: comparator_n.v

module comparator_n #(
    parameter WIDTH = 8
)(
    input wire [WIDTH-1:0] a, b,
    output wire      eq, // a == b
    output wire      gt, // a > b

```

```

        output wire      lt  // a < b
    );
    assign eq = (a == b);
    assign gt = (a > b);
    assign lt = (a < b);

```

```
endmodule
```

This is intentionally simple — Verilog's relational operators already synthesize to efficient comparator hardware. There is no benefit to writing this with gates manually. Know **when not to overengineer** — this is one of those times.

Testbench — N-bit Adder (with carry and overflow checking)

```
// FILE: tb_adder_n.v
```

```
`timescale 1ns/1ps
```

```
module tb_adder_n;
```

```
    parameter WIDTH = 4; // small width — easy to hand-verify every case
```

```
    reg [WIDTH-1:0] a, b;
```

```
    reg      cin;
```

```
    wire [WIDTH-1:0] sum;
```

```
    wire      cout;
```

```
    adder_n #(.WIDTH(WIDTH)) dut (
```

```
        .a(a), .b(b), .cin(cin),
```

```
        .sum(sum), .cout(cout)
```

```
    );
```



```
integer pass_count, fail_count;
```

```
task check;
```

```
    input [WIDTH-1:0] exp_sum;
```

```
    input      exp_cout;
```

```
    input [63:0] test_num;
```

```
begin
```

```
    #1;
```

```
    if (sum === exp_sum && cout === exp_cout) begin
```

```
        $display("PASS [Test %0d] a=%0d b=%0d cin=%b -> sum=%0d cout=%b",
```

```
            test_num, a, b, cin, sum, cout);
```

```
        pass_count = pass_count + 1;
```

```
    end else begin
```

```
        $display("FAIL [Test %0d] a=%0d b=%0d cin=%b -> sum=%0d(exp %0d) cout=%b(exp  
%b)",
```

```
            test_num, a, b, cin, sum, exp_sum, cout, exp_cout);
```

```
        fail_count = fail_count + 1;
```

```
    end
```

```
end
```

```
endtask
```

```
initial begin
```

```
    pass_count = 0; fail_count = 0;
```

```
// Simple addition, no carry expected
```

```
a=4'd2; b=4'd3; cin=0; check(4'd5, 1'b0, 1);
```

```
// Addition that exactly fills 4 bits, no overflow
```

```

a=4'd7; b=4'd8; cin=0; check(4'd15, 1'b0, 2);

// Addition that overflows 4 bits -> carry must be 1
// 15 + 1 = 16 -> doesn't fit in 4 bits -> wraps to 0, cout=1
a=4'd15; b=4'd1; cin=0; check(4'd0, 1'b1, 3);

// cin=1 pushes a result into overflow that wouldn't otherwise
a=4'd15; b=4'd0; cin=1; check(4'd0, 1'b1, 4);

// Max + Max -> definitely overflows
a=4'd15; b=4'd15; cin=1; check(4'd15, 1'b1, 5);

// Zero case
a=4'd0; b=4'd0; cin=0; check(4'd0, 1'b0, 6);

$display("-----");
$display("TOTAL: %0d passed, %0d failed", pass_count, fail_count);
if (fail_count == 0) $display("ALL TESTS PASSED");
else $display("FAILURE DETECTED");

$finish;

end

initial begin

    $dumpfile("adder_n.vcd");

    $dumpvars(0, tb_adder_n);

end

```

endmodule

Verify test 5 by hand before you run it — this is good practice: $15 + 15 + 1(\text{cin}) = 31$. In 5-bit wide math, $31 = 11111$. Lower 4 bits = $1111 = 15$. Bit 4 (the carry) = 1. That matches `check(4'd15, 1'b1, 5)`.

BLOCK 3: MINI TASK (5–8 min)

Your Task — Two Parts

Part 1: Write the testbench for `subtractor_n.v` (WIDTH=4). Test these cases and compute the expected diff and borrow **by hand first**, then write the check calls:

1. $a=8, b=3 \rightarrow$ no borrow ($a \geq b$)
2. $a=3, b=8 \rightarrow$ borrow occurs ($a < b$)
3. $a=0, b=0 \rightarrow$ no borrow
4. $a=5, b=5 \rightarrow$ no borrow, diff=0

For each case, work out: $\text{diff} = (a - b) \bmod 16$ and $\text{borrow} = 1$ if $a < b$ else 0.

Part 2 — answer in words, no code: In the adder testbench above, test 3 has `cout=1` (carry occurred). Would the **overflow flag** also be 1 for that same test case ($a=15, b=1, \text{cin}=0$, both unsigned values, sum wraps to 0)? Use the signed-interpretation rule I gave you: *same-sign operands producing a different-sign result = overflow*. Treat 1111 and 0001 as 4-bit signed numbers, work out their signed values, work out the result's signed value, and tell me if overflow fires.

Answer Part 2 first — that's the conceptual checkpoint before Day 6.